ASSIGNMENT-17.3

2303A52481

SINDHU KATAKAM

BATCH-41

Task 1 Employee Dataset Cleaning

PROMPT:

give data of 10 employees consists of salary designation and experience. then handle missing values in salary,designation,and experience. remove duplicate employee IDs. Standardize text values job titles in lowercase. encode categorical features like designation,employement_type. i need output as a clean dataset suitable for HR analytics and prediction tasks.

CODE:

```
import pandas as pd
import numpy as np

# 2. Create a dictionary with employee data, including missing values and duplicate EmployeeIDs
data = {
    'EmployeeID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 102], # Duplicate EmployeeID: 102
    'Salary': [60000, 75000, np.nan, 80000, 65000, 90000, 55000, np.nan, 70000, 82000],
    'Designation': ['Manager', 'Engineer', None, 'Analyst', 'Engineer', 'Manager', 'Technician', 'Ana
    'Experience': [5, 8, 3, np.nan, 6, 10, 2, 7, 4, 9],
    'EmploymentType': ['Full-time', 'Full-time', 'Part-time', 'Full-time', 'Contract', 'Full-time', '
}

# 3. Create the pandas DataFrame
df_employees = pd.DataFrame(data)

# 4. Handle Missing Values in 'Salary'
median_salary = df_employees['Salary'].median()
df_employees['Salary'] = df_employees['Salary'].fillna(median_salary)

# 5. Handle Missing Values in 'Experience'
median_experience = df_employees['Experience'].median()
df_employees['Experience'] = df_employees['Experience'].fillna(median_experience)

# 6. Handle Missing Values in 'Designation'
df_employees['Designation'] = df_employees['Designation'].fillna('Unknown')

# 7. Remove Duplicate Employee IDs
df_employees = df_employees.drop_duplicates(subset=['EmployeeID'], keep='first')
```

```python
# 8. Standardize Job Titles (Designation to lowercase)
df_employees['Designation'] = df_employees['Designation'].str.lower()

# 9. Encode Categorical Features
df_employees = pd.get_dummies(df_employees, columns=['Designation', 'EmploymentType'], drop_first=Fal

# 10. Display the first few rows of the cleaned DataFrame
print("First few rows of the cleaned DataFrame:")
print(df_employees.head())

# 11. Display a concise summary of the cleaned DataFrame
print("\nConcise summary of the cleaned DataFrame:")
df_employees.info()

# 12. Display  cleaned DataFrame
print("\nthe cleaned DataFrame:")
print(df_employees)
```

Output:

```
First few rows of the cleaned DataFrame:
   EmployeeID   Salary  Experience  Designation_analyst  Designation_engineer  \
0         101  60000.0         5.0                False                 False
1         102  75000.0         8.0                False                  True
2         103  72500.0         3.0                False                 False
3         104  80000.0         6.0                 True                 False
4         105  65000.0         6.0                False                  True

   Designation_manager  Designation_technician  Designation_unknown  \
0                 True                   False                False
1                False                   False                False
2                False                   False                 True
3                False                   False                False
4                False                   False                False

   EmploymentType_Contract  EmploymentType_Full-time  EmploymentType_Part-time
0                    False                      True                     False
1                    False                      True                     False
2                    False                     False                      True
3                    False                      True                     False
4                     True                     False                     False

Concise summary of the cleaned DataFrame:
<class 'pandas.core.frame.DataFrame'>
Index: 9 entries, 0 to 8
```

```
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype
...  ---  ------                    --------------  -----
 0   EmployeeID                9 non-null      int64
 1   Salary                    9 non-null      float64
 2   Experience                9 non-null      float64
 3   Designation_analyst       9 non-null      bool
 4   Designation_engineer      9 non-null      bool
 5   Designation_manager       9 non-null      bool
 6   Designation_technician    9 non-null      bool
 7   Designation_unknown       9 non-null      bool
 8   EmploymentType_Contract   9 non-null      bool
 9   EmploymentType_Full-time  9 non-null      bool
 10  EmploymentType_Part-time  9 non-null      bool
dtypes: bool(8), float64(2), int64(1)
memory usage: 360.0 bytes

the cleaned DataFrame:
   EmployeeID   Salary  Experience  Designation_analyst  Designation_engineer  \
0         101  60000.0         5.0                False                 False
1         102  75000.0         8.0                False                  True
2         103  72500.0         3.0                False                 False
3         104  80000.0         6.0                 True                 False
4         105  65000.0         6.0                False                  True
5         106  90000.0        10.0                False                 False
6         107  55000.0         2.0                False                 False
7         108  72500.0         7.0                 True                 False
8         109  70000.0         4.0                False                 False
```

```
   Designation_manager  Designation_technician  Designation_unknown  \
0                 True                   False                False
1                False                   False                False
2                False                   False                 True
3                False                   False                False
4                False                   False                False
5                 True                   False                False
6                False                    True                False
7                False                   False                False
8                 True                   False                False

   EmploymentType_Contract  EmploymentType_Full-time  EmploymentType_Part-time
0                    False                      True                     False
1                    False                      True                     False
2                    False                     False                      True
3                    False                      True                     False
4                     True                     False                     False
5                    False                      True                     False
6                    False                     False                      True
7                    False                      True                     False
8                    False                      True                     False
```

Justification:

Cleaning the employee dataset ensures the data is accurate and consistent. Filling missing values avoids incomplete analysis, removing duplicates prevents errors, and standardizing text keeps data uniform. Encoding categorical fields helps models understand the data. Overall, it makes the dataset reliable for HR analysis and predictions.

Task 2 – Student Performance Data Preprocessing

prompt

take 10 students data. preprocess the student performance data. convert exam data column into datetime format. extract the features like month ,semester and day.normalize total marks .detect and handle outliers in marks using IQR or Z-score . the output has to be a transformed dataset with timebased academic features and normalized marks.

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Define the initial student data dictionary
data_students = {
    'StudentID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'exam_date': ['2023-01-15', '2023-01-20', '2023-02-10', '2023-02-15', '2023-03-01', '2023-07-05',
    'total_marks': [85, 72, 90, 68, 78, 92, 65, 88, 70, 95]
}

# 2. Create a pandas DataFrame named df_students
df_students = pd.DataFrame(data_students)

# 3. Convert the exam_date column to datetime objects
df_students['exam_date'] = pd.to_datetime(df_students['exam_date'])

# 4. Extract the month and day from the exam_date column
df_students['month'] = df_students['exam_date'].dt.month
df_students['day'] = df_students['exam_date'].dt.day

# 5. Create a semester column
df_students['semester'] = df_students['month'].apply(lambda x: 'Semester 1' if 1 <= x <= 6 else 'Seme

# 6. and 7. Initialize and apply Min-Max Scaling to the total_marks column
scaler = MinMaxScaler()
df_students['normalized_marks'] = scaler.fit_transform(df_students[['total_marks']])
```

```
# 8. Calculate Q1, Q3, and IQR for the total_marks column
Q1 = df_students['total_marks'].quantile(0.25)
Q3 = df_students['total_marks'].quantile(0.75)
IQR = Q3 - Q1

# 9. Determine the lower and upper bounds for outlier detection
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# 10. Filter out the outliers from df_students
df_students_no_outliers = df_students[(df_students['total_marks'] >= lower_bound) & (df_students['tot

# 11. Display the df_students_no_outliers DataFrame
print("Final preprocessed DataFrame (without outliers):")
print(df_students_no_outliers)
```

Output:

```
Final preprocessed DataFrame (without outliers):
   StudentID  exam_date  total_marks  month  day    semester  normalized_marks
0          1 2023-01-15           85      1   15  Semester 1          0.666667
1          2 2023-01-20           72      1   20  Semester 1          0.233333
2          3 2023-02-10           90      2   10  Semester 1          0.833333
3          4 2023-02-15           68      2   15  Semester 1          0.100000
4          5 2023-03-01           78      3    1  Semester 1          0.433333
5          6 2023-07-05           92      7    5  Semester 2          0.900000
6          7 2023-08-12           65      8   12  Semester 2          0.000000
7          8 2023-09-20           88      9   20  Semester 2          0.766667
8          9 2023-11-01           70     11    1  Semester 2          0.166667
9         10 2023-12-15           95     12   15  Semester 2          1.000000
```

Justification

The purpose of this task was to clean and prepare student performance data so it can be easily used for analysis. We first converted exam dates into a format that allowed us to extract useful information like the month, day, and semester. Then, we adjusted the total marks so they are all on a consistent scale, which is important for fair comparisons. Finally, we identified and addressed any unusual or extreme marks that could mislead our analysis. All these steps were then put into a single piece of code for convenience and clarity, making the data ready for further study.

Task 3 – Student Health Data Preparation

Prompt

give student health data . the data has to be clean and process the student health records using AI Scripts. handle missing values in height,weight,BMI . Encode categorical features like medical_condition,fitness status. scale the numerical features using standard scaling. split dataset into training and testing sets. the output has to be clean and neat dataset ready for predictive health analysis. give output as a table and give the complete code in single cell

```python
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# 1. Generate Sample Student Health Data
np.random.seed(42)
num_students = 100

data = {
    'student_id': range(1, num_students + 1),
    'age': np.random.randint(18, 25, num_students),
    'height': np.random.normal(170, 10, num_students), # cm
    'weight': np.random.normal(70, 15, num_students), # kg
    'medical_condition': np.random.choice(['None', 'Asthma', 'Diabetes', 'Allergies'], num_students, p=[0.7, 0.1, 0.1, 0.1]),
    'fitness_status': np.random.choice(['Excellent', 'Good', 'Average', 'Poor'], num_students, p=[0.25, 0.35, 0.3, 0.1])
}

df = pd.DataFrame(data)
```

```python
# Introduce some missing values randomly
for col in ['height', 'weight']:
    missing_indices = np.random.choice(df.index, size=int(num_students * 0.05), replace=False)
    df.loc[missing_indices, col] = np.nan

# Calculate BMI (weight in kg / (height in meters)^2)
df['height_m'] = df['height'] / 100
df['BMI'] = df['weight'] / (df['height_m'] ** 2)

# Introduce missing BMI values directly (if not already missing due to height/weight)
missing_bmi_indices = np.random.choice(df.index, size=int(num_students * 0.03), replace=False)
df.loc[missing_bmi_indices, 'BMI'] = np.nan

print("Original DataFrame head with introduced missing values:")
display(df.head())
print("\nMissing values before processing:")
display(df.isnull().sum())

# 2. Define preprocessing steps
# Numerical features to impute and scale
numerical_features = ['age', 'height', 'weight', 'BMI']

# Categorical features to encode
categorical_features = ['medical_condition', 'fitness_status']
```

```python
# Create a column transformer for preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='mean')),
            ('scaler', StandardScaler())
        ]), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ],
    remainder='passthrough' # Keep other columns (like student_id) as they are
)

# 3. Apply preprocessing to the DataFrame
# Fit and transform the data
X_processed = preprocessor.fit_transform(df)

# Get feature names after one-hot encoding
# This requires knowing the categories generated by OneHotEncoder
# Let's get them from the fitted preprocessor
cat_encoder = preprocessor.named_transformers_['cat']
cat_feature_names = cat_encoder.get_feature_names_out(categorical_features)

processed_feature_names = numerical_features + list(cat_feature_names)
```

```
X_train, X_test = train_test_split(df_processed, test_size=0.2, random_state=42)

print("\nShape of training set:", X_train.shape)
print("Shape of testing set:", X_test.shape)

print("\nFirst 5 rows of the training set:")
display(X_train.head())

print("\nFirst 5 rows of the testing set:")
display(X_test.head())
```

OUTPUT

```
Original DataFrame head with introduced missing values:
```

	student_id	age	height	weight	medical_condition	fitness_status	height_m	BMI
0	1	24	168.843517	45.887751	None	Poor	1.688435	16.096372
1	2	21	166.988963	72.769508	Allergies	Good	1.669890	26.095998
2	3	22	155.214780	73.898242	Diabetes	Average	1.552148	30.673827
3	4	24	162.801558	81.727343	None	Good	1.628016	30.835448
4	5	20	165.393612	51.445739	None	Good	1.653936	18.806672

```
Missing values before processing:
```

	0
student_id	0
age	0
height	5
weight	5
medical_condition	0
fitness_status	0
height_m	5
BMI	13

```
Processed DataFrame head:
```

	age	height	weight	BMI	medical_condition_Allergies	medical_condition_Asthma	medical_condition_Diabetes	medical_condition_None	fitness_status_Average	f:
0	1.465198	-0.107598	-1.762091	-1.627874	0.0	0.0	0.0	1.0	0.0	
1	-0.110284	-0.310790	0.103548	0.230905	1.0	0.0	0.0	0.0	0.0	
2	0.414877	-1.600814	0.181885	1.081855	0.0	0.0	1.0	0.0	1.0	
3	1.465198	-0.769578	0.725237	1.111898	0.0	0.0	0.0	1.0	0.0	
4	-0.635444	-0.485583	-1.376357	-1.124070	0.0	0.0	0.0	1.0	0.0	

```
Missing values after processing:
```

	0
age	0
height	0
weight	0
BMI	0
medical_condition_Allergies	0
medical_condition_Asthma	0
medical_condition_Diabetes	0
medical_condition_None	0
fitness_status_Average	0
fitness_status_Excellent	0
fitness_status_Good	0
fitness_status_Poor	0
student_id	0
height_m	5

```
dtype: int64

Shape of training set: (80, 14)
Shape of testing set: (20, 14)

First 5 rows of the training set:
```

	age	height	weight	BMI	medical_condition_Allergies	medical_condition_Asthma	medical_condition_Diabetes	medical_condition_None	fitness_status_Average	f
55	1.465198	0.305132	0.071372	-0.108841	1.0	0.0	0.0	0.0	1.0	
88	0.414877	-1.679855	-0.848990	-0.021577	0.0	0.0	0.0	1.0	1.0	
26	-0.110284	0.415333	-0.344115	-0.530148	0.0	0.0	0.0	1.0	1.0	
42	1.465198	-0.866706	-0.321291	0.105655	0.0	0.0	0.0	1.0	0.0	
69	1.465198	-0.009939	1.093493	0.995553	0.0	0.0	0.0	1.0	1.0	

```
First 5 rows of the testing set:
```

	age	height	weight	BMI	medical_condition_Allergies	medical_condition_Asthma	medical_condition_Diabetes	medical_condition_None	fitness_status_Average	f
83	0.940037	2.419057	1.044965	-0.262253	1.0	0.0	0.0	0.0	0.0	
53	0.414877	-1.584372	0.312465	1.215738	0.0	0.0	0.0	1.0	0.0	
70	-0.110284	0.085101	0.904480	0.764655	0.0	0.0	0.0	1.0	0.0	
45	-0.110284	0.379303	-0.164477	-0.354382	0.0	0.0	0.0	1.0	0.0	
44	-0.635444	1.022060	0.403992	-0.154784	0.0	0.0	0.0	1.0	0.0	

Justification:

The preprocessing steps were crucial to get your student health data ready for analysis. First, we filled in any missing information in height, weight, and BMI by using the average values, which helps to avoid errors in our calculations. Then, we converted text categories like 'medical condition' and 'fitness status' into a numerical format that computers can understand, while also making sure all other numerical data like age and height were scaled uniformly. This standardization prevents features with larger values from unfairly dominating the analysis. Finally, we split the data into training and testing sets, which is a standard practice to evaluate how well a future predictive model performs on unseen data.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Prompt:

give student health data . the data has to be clean and process the student health records using AI Scripts. handle missing values in height,weight,BMI . Encode categorical features like medical_condition,fitness status. scale the numerical features using standard scaling. split dataset into training and testing sets. the output has to be clean and neat dataset ready for predictive health analysis.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# --- 1. Generate Sample Ride-Sharing Trip Data with Errors ---
np.random.seed(42)
num_trips = 100

data = {
    'trip_id': range(1001, 1001 + num_trips),
    'pickup_location': np.random.choice([
        'Downtown', 'Airport', 'Suburbia ', 'University Campus', '  Mall  ',
        'DOWNTOWN', 'AirPort', 'Suburbia', 'University Campus'
    ], num_trips),
    'drop_location': np.random.choice([
        'Downtown', 'Airport', 'Suburbia', 'University Campus', 'Mall',
        'DOWNTOWN', 'AirPort', 'Suburbia ', 'University Campus'
    ], num_trips),
    'fare_amount': np.random.normal(25, 10, num_trips),
    'driver_rating': np.random.choice([1, 2, 3, 4, 5, np.nan], num_trips, p=[0.05, 0.05, 0.1, 0.3, 0.4, 0.1]),
    'pickup_time': pd.to_datetime('2023-01-01') + pd.to_timedelta(np.arange(num_trips), unit='h')
}
```

```
df = pd.DataFrame(data)

# Introduce some missing fare amounts
missing_fare_indices = np.random.choice(df.index, size=int(num_trips * 0.05), replace=False)
df.loc[missing_fare_indices, 'fare_amount'] = np.nan

# Introduce some duplicate entries
duplicate_rows = df.sample(5, random_state=42)
df = pd.concat([df, duplicate_rows], ignore_index=True)

print("Original DataFrame head with introduced errors:")
display(df.head())
print("\nOriginal Missing values:\n", df.isnull().sum())
print("\nOriginal Number of duplicate entries:", df.duplicated().sum())
print("\nOriginal Driver Rating range:", df['driver_rating'].min(), '-', df['driver_rating'].max())
print("\n------------------------------------------------------------")

# --- 2. Clean and Prepare the Data ---

# Store initial metrics for summary
initial_missing_fares = df['fare_amount'].isnull().sum()
initial_duplicates = df.duplicated(subset=['trip_id', 'pickup_time']).sum()

# a. Standardize pickup and drop locations
```

```python
# a. Standardize pickup and drop locations
print("\nStandardizing pickup and drop locations...")
df['pickup_location'] = df['pickup_location'].str.strip().str.lower()
df['drop_location'] = df['drop_location'].str.strip().str.lower()
print("Locations standardized (stripped whitespace, converted to lowercase).")

# b. Fill missing fare amounts with median values
print("\nFilling missing fare amounts...")
median_fare = df['fare_amount'].median()
df['fare_amount'] = df['fare_amount'].fillna(median_fare)
filled_fares = df['fare_amount'].isnull().sum() - initial_missing_fares # This will be negative
print(f"Missing fare amounts filled with median ({median_fare:.2f}). {initial_missing_fares} values imputed.")

# c. Remove duplicate ride entries (based on trip_id and pickup_time for robustness)
print("\nRemoving duplicate ride entries...")
df.drop_duplicates(subset=['trip_id', 'pickup_time'], inplace=True)
duplicates_removed = initial_duplicates - df.duplicated(subset=['trip_id', 'pickup_time']).sum()
print(f"{duplicates_removed} duplicate entries removed.")

# d. Normalize driver ratings to a 0-1 scale
print("\nNormalizing driver ratings...")
# Handle potential NaN values before scaling, using median for consistency
df['driver_rating'] = df['driver_rating'].fillna(df['driver_rating'].median())
```

```python
scaler = MinMaxScaler(feature_range=(0, 1))
df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])
print("Driver ratings normalized to 0-1 scale.")

print("\n------------------------------------------------------------")
print("--- Summary of Data Improvements ---")
print(f"- {initial_missing_fares} missing 'fare_amount' values filled with median.")
print(f"- {duplicates_removed} duplicate ride entries removed.")
print(f"- 'pickup_location' and 'drop_location' standardized to lowercase and stripped.")
print(f"- 'driver_rating' normalized to a 0-1 scale (missing values imputed with median before normalization).")

print("\nCleaned DataFrame head:")
display(df.head())
print("\nMissing values after processing:\n", df.isnull().sum())
print("\nNumber of duplicate entries after processing:", df.duplicated().sum())
print("\nDriver Rating (Normalized) range:", df['driver_rating_normalized'].min(), '-', df['driver_rating_normalized'].max())
```

OUTPUT:

```
Original DataFrame head with introduced errors:
```

	employee_id	age	gender	employment_type	designation	experience_years	salary
0	101	50	Male	Full-time	Project Manager	2.0	41038.313170
1	102	36	Male	Full-time	Software Engineer	16.0	41850.724512
2	103	29	Female	Part-time	Software Engineer	NaN	55631.115575
3	104	42	Female	Contractor	Intern	15.0	65731.056966
4	105	40	Female	Full-time	Data Analyst	3.0	76218.151312

```
Missing values before cleaning:
```

	0
employee_id	0
age	0
gender	0
employment_type	0
designation	5
experience_years	6
salary	10

```
dtype: int64

Number of duplicate employee_ids before cleaning: 5

Unique designations before cleaning: ['Project Manager' 'Software Engineer' 'Intern' 'Data Analyst' nan
 'Marketing Manager' 'HR Specialist' 'SOFTWARE ENGINEER']
```

JUSTIFICATION:

The task of cleaning and preparing the ride-sharing trip data is crucial for several reasons. Firstly, standardizing pickup and drop locations ensures consistency and accuracy, preventing issues caused by varied spellings or formatting. Secondly, filling missing fare amounts with a median value helps to retain valuable data points that would otherwise be discarded, allowing for a more complete analysis without skewing financial metrics. Thirdly, removing duplicate ride entries eliminates redundant information, ensuring that each trip is counted only once and preventing inflated statistics. Lastly, normalizing driver ratings allows for fair comparison and analysis across different drivers, as it brings all ratings to a common scale. Together, these steps transform raw, error-prone data into a reliable and structured dataset, enabling accurate operational analysis and informed business decisions for the ride-sharing company.